```
!pip install datasets transformers torch torchvision matplotlib -q

from datasets import load_dataset
dataset = load_dataset("huggan/wikiart", split="train[:500]")
print(f"Loaded{len(dataset)} paintings")
print("columns:", dataset.column_names)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```

{"model_id":"70aafb75df594aedbf2735cfc7dde58d","version_major":2,"version_minor":0}

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

{"model_id":"637fea32948e4782a667167917b096d8","version_major":2,"version_minor":0}

{"model_id":"95fc2941babd4e749a71a18544a4215a","version_major":2,"version_minor":0}

{"model_id":"70871d7bf528489abcdf5b53d21b38e8","version_major":2,"version_minor":0}

{"model_id":"e48db4e00cf54eaa9a29f366139caeb9","version_major":2,"version_minor":0}

{"model_id":"731b00b5827846fc932e2ddd3709f1a4","version_major":2,"version_minor":0}

{"model_id":"cd34eda5dda641b282ad80d062dfa9ff","version_major":2,"version_minor":0}

{"model_id":"ef8c7ffee96549399ff5c8d3d2fb1fbd","version_major":2,"version_minor":0}

{"model_id":"2abc27fcb8dc4fa88013384f5c1be64a","version_major":2,"version_minor":0}

{"model_id":"6df99651cbdc408facd1ea67e7b80989","version_major":2,"version_minor":0}

{"model_id":"b94f4824978245e58ac3da1170c9d50e","version_major":2,"version_minor":0}

{"model_id":"c07e04639ce6447f8fd5bc338f801def","version_major":2,"version_minor":0}

{"model_id":"7f59e9451d474d429a1a5ea1b3afcb52","version_major":2,"version_minor":0}

{"model_id":"4e116a6aeea74fa696289be04c7f42bc","version_major":2,"version_minor":0}

{"model_id":"77b170d3e9a0454c98b40b5940fd48c3","version_major":2,"version_minor":0}

{"model_id":"b3fd4f530236484895e35e28a27ff3e3","version_major":2,"version_minor":0}

{"model_id":"75975dde6c184bc1a617219c24445c50","version_major":2,"version_minor":0}

{"model_id":"45c5d73f189a43c880efa7fbd0dc4ec5","version_major":2,"version_minor":0}

{"model_id":"ca7ece5db07a47a3bf344cac88324d92","version_major":2,"version_minor":0}

{"model_id":"fe90bb1985dd429ea84ccd18bd85001d","version_major":2,"version_minor":0}

{"model_id":"0f33ac3181734dfeaf2c665eb23181a6","version_major":2,"version_minor":0}

{"model_id":"3a51bcfa631f4457b6b0d3396dcc126b","version_major":2,"version_minor":0}

{"model_id":"314ec3e02f5c46119cf8c3d23071bb40","version_major":2,"version_minor":0}

{"model_id":"2731722a48b14baeade815d75ebddafb","version_major":2,"version_minor":0}

{"model_id":"36a0a06ecb524c769471471b5dc629c9","version_major":2,"version_minor":0}

{"model_id":"aae2c3f2bb544e54900ae7eece4c29b0","version_major":2,"version_minor":0}

{"model_id":"af2879b0d50b4553b3ca47a8929472a2","version_major":2,"version_minor":0}

{"model_id":"4d6e7127e74e4ba08f605facf0e98f64","version_major":2,"version_minor":0}

{"model_id":"69f8e82eb740427583e4035efeae89a1","version_major":2,"version_minor":0}

{"model_id":"8907804627a04bbb9ce1fc90abe2bccc","version_major":2,"version_minor":0}

{"model_id":"5e24cd360bb2479882e36c6218303ac1","version_major":2,"version_minor":0}

{"model_id":"534267e9877647cdb050c94d08ffa0b2","version_major":2,"version_minor":0}

{"model_id":"821f93799c434a8cad5a907075a9da2b","version_major":2,"version_minor":0}

{"model_id":"c555d909828b4a2886c0f6beff6feea5","version_major":2,"version_minor":0}

{"model_id":"c7e87cd1905c4bbc8349555eb1e929c8","version_major":2,"version_minor":0}

{"model_id":"9070063107ea40a09a249875e0f57344","version_major":2,"version_minor":0}

{"model_id":"94b121618a1144db84dd157fa4b96860","version_major":2,"version_minor":0}

{"model_id":"a88f4604780844b18689f4f2a5ceaa6d","version_major":2,"version_minor":0}

{"model_id":"98dd23a4e8544b2b80b87395099fc708","version_major":2,"version_minor":0}

{"model_id":"4ede751ed25a46a28a0a49358a35f2e3","version_major":2,"version_minor":0}

{"model_id":"8004c593d5e24f5ca9772aa8787c309e","version_major":2,"version_minor":0}

{"model_id":"aa86564028634071859a108ac16a37c0","version_major":2,"version_minor":0}

{"model_id":"23f36041a1a04056abc775ce317bdea7","version_major":2,"version_minor":0}

{"model_id":"bbaa77855d2f43b187b842bae2c6b379","version_major":2,"version_minor":0}

{"model_id":"e60f8297d0204d2598e63b25f319b109","version_major":2,"version_minor":0}

{"model_id":"246aad8ff2fe42df870d53fc09bb4792","version_major":2,"version_minor":0}

{"model_id":"f07f8ae54c3d40949ea10e1d1a2c041b","version_major":2,"version_minor":0}

{"model_id":"9b2122a7bcd5404481a921567306f982","version_major":2,"version_minor":0}

{"model_id":"f9265686aa0f48ddacc750f348b4149b","version_major":2,"version_minor":0}

{"model_id":"088f25b652c94d7ea74216cb13262607","version_major":2,"version_minor":0}

{"model_id":"33c4787f29e34d95b0e921813419d121","version_major":2,"version_minor":0}

{"model_id":"abb0aae6da604c9d8e50fe2d0a578d05","version_major":2,"version_minor":0}

{"model_id":"7e40e88df3084e97b5e9a5a3b4e2d7c8","version_major":2,"version_minor":0}

{"model_id":"31be72a3daf441d89854724090f6a081","version_major":2,"version_minor":0}

{"model_id":"2202764084914d2aa8bfc005437fce25","version_major":2,"version_minor":0}

{"model_id":"64ee7732d90b4bc0921b14235105e1aa","version_major":2,"version_minor":0}

{"model_id":"c1cf3e6a5d934961aacfd0fee221ffc1","version_major":2,"version_minor":0}

{"model_id":"5dd54807182c4511823d5975000a76e3","version_major":2,"version_minor":0}

{"model_id":"fdf754f799a74752b0a91c0bfa064fd9","version_major":2,"version_minor":0}

{"model_id":"8536b7e10e1b47a4b443f236fc22c88c","version_major":2,"version_minor":0}

{"model_id":"269619a91db448729f7c345b012d0408","version_major":2,"version_minor":0}

{"model_id":"03582af369e040a288b2ef571072dd36","version_major":2,"version_minor":0}

{"model_id":"61b9f6747f84457b9edd314049d075f7","version_major":2,"version_minor":0}

{"model_id":"a4bb543a78854db39d26625f2a604558","version_major":2,"version_minor":0}

{"model_id":"3c8236221ca7406ba6587b513a393786","version_major":2,"version_minor":0}

{"model_id":"edc9a665ebe34e058932ed3db4aada84","version_major":2,"version_minor":0}

{"model_id":"6d6cc6c4bdc145cc99918fd42082012b","version_major":2,"version_minor":0}

{"model_id":"f8efa242d2954d2994b772f1c557313f","version_major":2,"version_minor":0}

{"model_id":"923b8ea23ee94aa6b20cf48b27fe53d2","version_major":2,"version_minor":0}

{"model_id":"b9911edb71ba426fbeb0c281bad8271d","version_major":2,"version_minor":0}

{"model_id":"5da4de8fe4be460ca7796c7a82050823","version_major":2,"version_minor":0}

{"model_id":"0f1abdf847a947ffa1e17e614bd4c2b6","version_major":2,"version_minor":0}

{"model_id":"fb4e24bf19f343d88ca79bd2615b3778","version_major":2,"version_minor":0}

{"model_id":"636c72a139964f02b9bf0bd7f5ce8cf2","version_major":2,"version_minor":0}

{"model_id":"db798c2d388d432e863a670c6ada0825","version_major":2,"version_minor":0}

{"model_id":"838af2bfa172482d9bfa4e0e9fd06659","version_major":2,"version_minor":0}

{"model_id":"07e8b39749974b4fada5a9c88f023e09","version_major":2,"version_minor":0}

```
Loaded500 paintings
columns: ['image', 'artist', 'genre', 'style']
```

```
import os
os.system("pip cache purge")
!rm -rf /root/.cache/huggingface/datasets/downloads
```

```
!df -h /
```

```
Filesystem      Size  Used Avail Use% Mounted on
overlay         113G  107G  6.3G  95% /
```

```
!rm -rf /root/.cache/huggingface/hub
!rm -rf /tmp/*
!df -h /
```

```
Filesystem      Size  Used Avail Use% Mounted on
overlay         113G   75G   38G  67% /
```

```
from datasets import load_dataset

dataset = load_dataset("huggan/wikiart", split="train[:500]")
print(f"Loaded {len(dataset)} paintings")
print("Columns:", dataset.column_names)
```

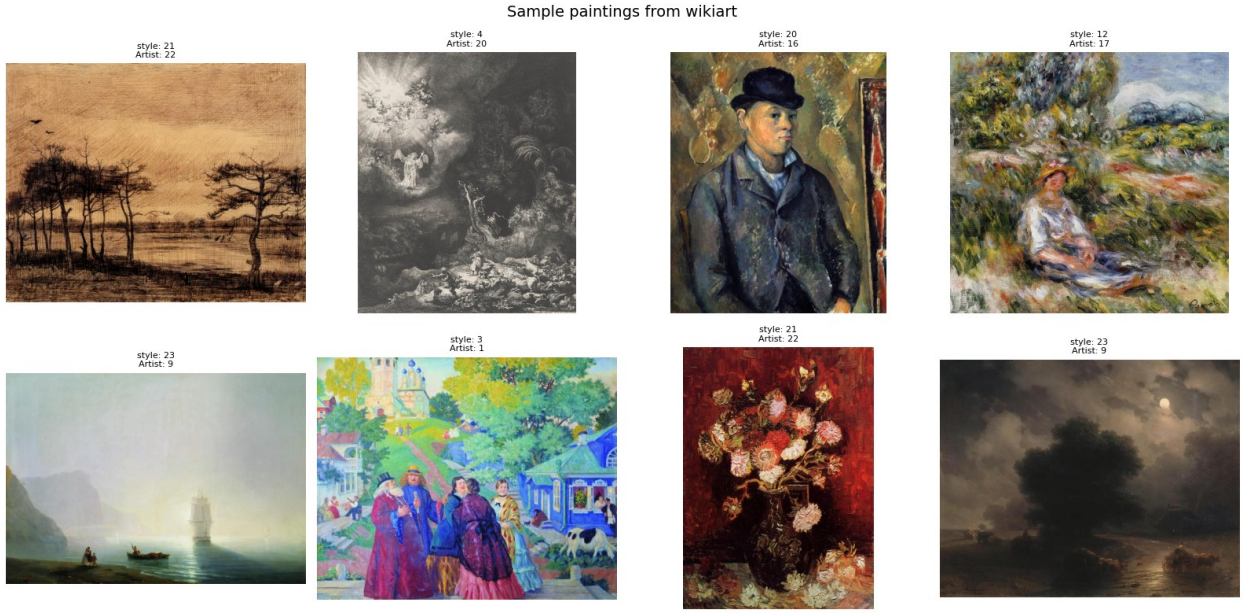
{"model_id":"72bc4189bf274c218880ecd53944c82e","version_major":2,"version_minor":0}

{"model_id":"d770d7a5304048b7b09c9b154140a948","version_major":2,"version_minor":0}

{"model_id":"bb7467de10e746e6a25b52c14916ebf1","version_major":2,"version_minor":0}

```
Loaded 500 paintings
Columns: ['image', 'artist', 'genre', 'style']
```

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2,4, figsize=(16,8))
for i, ax in enumerate(axes.flat):
    item = dataset[i]
    ax.imshow(item["image"])
    ax.set_title(f"style: {item['style']}\nArtist: {item['artist']}",
fontsize=8)
    ax.axis("off")

plt.suptitle("Sample paintings from wikiart", fontsize=14)
plt.tight_layout()
plt.show()
```

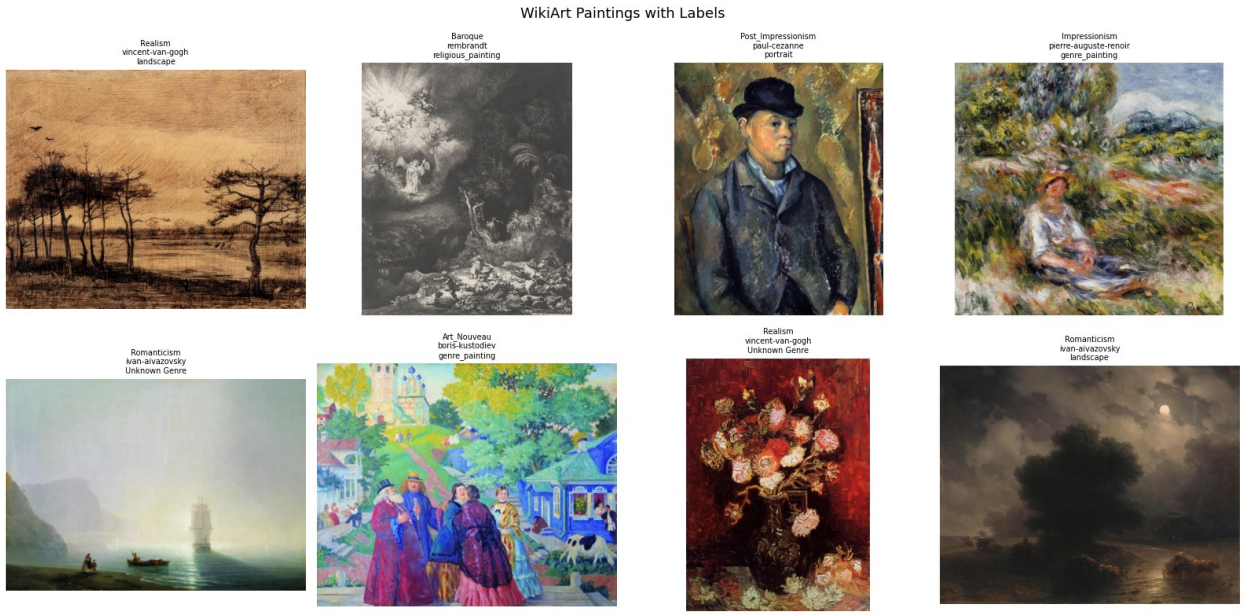



```python
print("Style labels:")
print(dataset.features["style"].names)
print("\nArtist labels:")
print(dataset.features["artist"].names)
print("\nGenre labels:")
print(dataset.features["genre"].names)
```

```
Style labels:
['Abstract_Expressionism', 'Action_painting', 'Analytical_Cubism',
'Art_Nouveau', 'Baroque', 'Color_Field_Painting',
'Contemporary_Realism', 'Cubism', 'Early_Renaissance',
'Expressionism', 'Fauvism', 'High_Renaissance', 'Impressionism',
'Mannerism_Late_Renaissance', 'Minimalism', 'Naive_Art_Primitivism',
'New_Realism', 'Northern_Renaissance', 'Pointillism', 'Pop_Art',
'Post_Impressionism', 'Realism', 'Rococo', 'Romanticism', 'Symbolism',
'Synthetic_Cubism', 'Ukiyo_e']

Artist labels:
['Unknown Artist', 'boris-kustodiev', 'camille-pissarro', 'childe-
hassam', 'claude-monet', 'edgar-degas', 'eugene-boudin', 'gustave-
dore', 'ilya-repin', 'ivan-aivazovsky', 'ivan-shishkin', 'john-singer-
sargent', 'marc-chagall', 'martiros-saryan', 'nicholas-roerich',
'pablo-picasso', 'paul-cezanne', 'pierre-auguste-renoir', 'pyotr-
konchalovsky', 'raphael-kirchner', 'rembrandt', 'salvador-dali',
'vincent-van-gogh', 'hieronymus-bosch', 'leonardo-da-vinci',
'albrecht-durer', 'edouard-cortes', 'sam-francis', 'juan-gris',
'lucas-cranach-the-elder', 'paul-gauguin', 'konstantin-makovsky',
'egon-schiele', 'thomas-eakins', 'gustave-moreau', 'francisco-goya',
'edvard-munch', 'henri-matisse', 'fra-angelico', 'maxime-maufra',
'jan-matejko', 'mstislav-dobuzhinsky', 'alfred-sisley', 'mary-
cassatt', 'gustave-loiseau', 'fernando-botero', 'zinaida-
```

```
serebriakova', 'georges-seurat', 'isaac-levitan', 'joaquã\xadn-
sorolla', 'jacek-malczewski', 'berthe-morisot', 'andy-warhol',
'arkhip-kuindzhi', 'niko-pirosmani', 'james-tissot', 'vasily-polenov',
'valentin-serov', 'pietro-perugino', 'pierre-bonnard', 'ferdinand-
hodler', 'bartolome-esteban-murillo', 'giovanni-boldini', 'henri-
martin', 'gustav-klimt', 'vasily-perov', 'odilon-redon', 'tintoretto',
'gene-davis', 'raphael', 'john-henry-twachtman', 'henri-de-toulouse-
lautrec', 'antoine-blanchard', 'david-burliuk', 'camille-corot',
'konstantin-korovin', 'ivan-bilibin', 'titian', 'maurice-prendergast',
'edouard-manet', 'peter-paul-rubens', 'aubrey-beardsley', 'paolo-
veronese', 'joshua-reynolds', 'kuzma-petrov-vodkin', 'gustave-
caillebotte', 'lucian-freud', 'michelangelo', 'dante-gabriel-
rossetti', 'felix-vallotton', 'nikolay-bogdanov-belsky', 'georges-
braque', 'vasily-surikov', 'fernand-leger', 'konstantin-somov',
'katsushika-hokusai', 'sir-lawrence-alma-tadema', 'vasily-
vereshchagin', 'ernst-ludwig-kirchner', 'mikhail-vrubel', 'orest-
kiprensky', 'william-merritt-chase', 'aleksey-savrasov', 'hans-
memling', 'amedeo-modigliani', 'ivan-kramskoy', 'utagawa-kuniyoshi',
'gustave-courbet', 'william-turner', 'theo-van-rysselberghe', 'joseph-
wright', 'edward-burne-jones', 'koloman-moser', 'viktor-vasnetsov',
'anthony-van-dyck', 'raoul-dufy', 'frans-hals', 'hans-holbein-the-
younger', 'ilya-mashkov', 'henri-fantin-latour', 'm.c.-escher', 'el-
greco', 'mikalojus-ciurlionis', 'james-mcneill-whistler', 'karl-
bryullov', 'jacob-jordaens', 'thomas-gainsborough', 'eugene-
delacroix', 'canaletto']

Genre labels:
['abstract_painting', 'cityscape', 'genre_painting', 'illustration',
'landscape', 'nude_painting', 'portrait', 'religious_painting',
'sketch_and_study', 'still_life', 'Unknown Genre']
```

```python
fig, axes = plt.subplots(2, 4, figsize=(16, 8))

style_names  = dataset.features["style"].names
artist_names = dataset.features["artist"].names
genre_names  = dataset.features["genre"].names

for i, ax in enumerate(axes.flat):
    item = dataset[i]
    style  = style_names[item["style"]]
    artist = artist_names[item["artist"]]
    genre  = genre_names[item["genre"]]
    ax.imshow(item["image"])
    ax.set_title(f"{style}\n{artist}\n{genre}", fontsize=7)
    ax.axis("off")

plt.suptitle("WikiArt Paintings with Labels", fontsize=13)
plt.tight_layout()
plt.show()
```



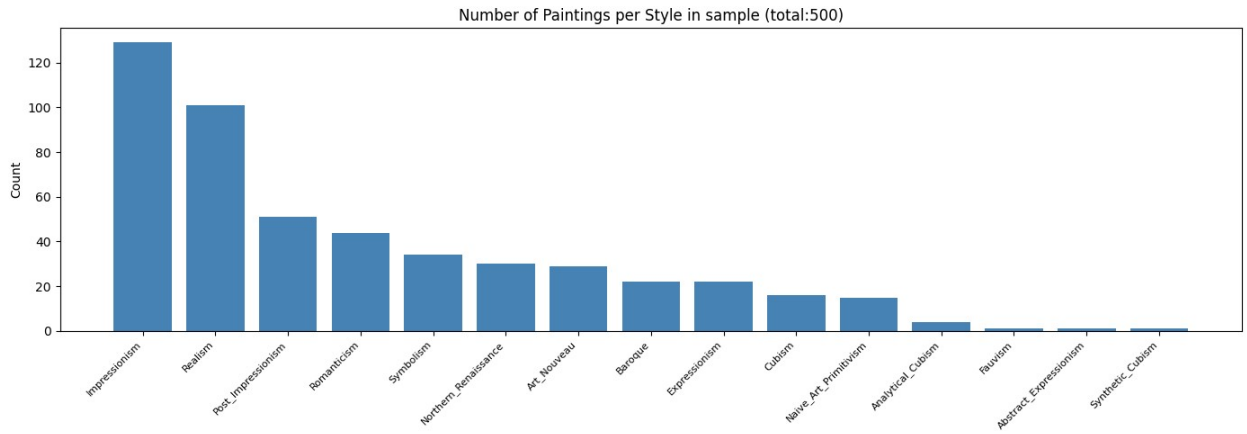
MY OBSERVATIONS: (from above 8 paintings)

1. Realism (van gogh) and romanticism share similar ladscape colors so model may confuse between these two
2. van gogh's realism flower painting and post impressionism by paul cez share similar brush work and strokes
3. Impressionaism and Art noveu female figures drawn with similar softness
4. van gogh in realism surprised me as these works being different from his popular starry night like spiral brush strokes suggesting artist labels dont tell the full visual story and it can be used as in outlier.

below: Checking for class imbalance so a lazy model doesnt give the max frequency one as prediction all time

```
from collections import Counter

style_counts = Counter(dataset[i]["style"] for i in
range(len(dataset)))


named_counts = {style_names[k]: v for k, v in style_counts.items()}
named_counts = dict(sorted(named_counts.items(), key=lambda x: -x[1]))

plt.figure(figsize=(14, 5))
plt.bar(named_counts.keys(), named_counts.values(), color="steelblue")
plt.xticks(rotation=45, ha="right", fontsize=8)
plt.title("Number of Paintings per Style in sample (total:500)")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
print("\nTop 5 most common styles:")
for style, count in list(named_counts.items())[:5]:
    print(f"  {style}: {count}")

print("\nBottom 5 least common styles:")
for style, count in list(named_counts.items())[-5:]:
    print(f"  {style}: {count}")
```




```
Top 5 most common styles:
  Impressionism: 129
  Realism: 101
  Post_Impressionism: 51
  Romanticism: 44
  Symbolism: 34

Bottom 5 least common styles:
  Naive_Art_Primitivism: 15
  Analytical_Cubism: 4
  Fauvism: 1
  Abstract_Expressionism: 1
  Synthetic_Cubism: 1
```

Class imbalance problem: Impressionism dominating 129 paintings whereas, fauvism, synthetic cubism and abstract expressionism have 1 painting each only in 500 sample aka if model predicts impressionism for every painting it is right almost 25% time without learning, therefore accuracy is not preferred and need to use weighted loss

```
import torch
import torchvision.models as models
import torch.nn as nn

# Loading ResNet50 - removing the final classification layer
backbone = models.resnet50(weights="IMAGENET1K_V2")
# Removing the last fully connected layer
```

```
features = nn.Sequential(*list(backbone.children())[:-2])

# Testing
fake_image = torch.randn(1, 3, 224, 224)

with torch.no_grad():
    output = features(fake_image)

print(f"Input shape:  {fake_image.shape}")
print(f"Output shape: {output.shape}")
print(f"\nMeaning of output shape:")
print(f"  {output.shape[0]} = number of images in batch")
print(f"  {output.shape[1]} = number of feature channels (2048 learned
filters)")
print(f"  {output.shape[2]}x{output.shape[3]} = spatial grid
({output.shape[2]*output.shape[3]} patches)")
```

```
Downloading: "https://download.pytorch.org/models/resnet50-
11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-
11ad3fa6.pth

100%|██████████| 97.8M/97.8M [00:00<00:00, 183MB/s]

Input shape:  torch.Size([1, 3, 224, 224])
Output shape: torch.Size([1, 2048, 7, 7])

Meaning of output shape:
  1 = number of images in batch
  2048 = number of feature channels (2048 learned filters)
  7x7 = spatial grid (49 patches)
```

our test oainting went as 224X224 pixel but got divided into 7X7 grids or 49 patches, each oatch beinf read and studied separately so if we read each patch and store then compare with others we can classify based on patch similarity

```
# Reshaping from 2d to 1d for ltsm
batch_size, channels, h, w = output.shape

# Flatten spatial dimensions: [1, 2048, 7, 7] → [1, 2048, 49]
sequence = output.flatten(start_dim=2)

# Transpose: [1, 2048, 49] → [1, 49, 2048]
sequence = sequence.permute(0, 2, 1)

print(f"Before reshape: {output.shape}")
print(f"After reshape:  {sequence.shape}")
print(f"\nMeaning: {sequence.shape[0]} image, {sequence.shape[1]}
patches in sequence, {sequence.shape[2]} features per patch")
```

```
Before reshape: torch.Size([1, 2048, 7, 7])
After reshape:  torch.Size([1, 49, 2048])

Meaning: 1 image, 49 patches in sequence, 2048 features per patch
```

```
# ltsm fast
projection = nn.Linear(2048, 512)
sequence_small = projection(sequence)

print(f"Before projection: {sequence.shape}")
print(f"After projection:  {sequence_small.shape}")
```

```
Before projection: torch.Size([1, 49, 2048])
After projection:  torch.Size([1, 49, 512])
```

```
# Bidirectional LSTM
lstm = nn.LSTM(
    input_size=512,
    hidden_size=256,
    num_layers=2,
    batch_first=True,
    bidirectional=True
)

with torch.no_grad():
    lstm_out, _ = lstm(sequence_small)

print(f"Input shape:  {sequence_small.shape}")
print(f"Output shape: {lstm_out.shape}")
```

```
Input shape:  torch.Size([1, 49, 512])
Output shape: torch.Size([1, 49, 512])
```

```
# Attention
attention_weights = torch.softmax(lstm_out.mean(dim=-1), dim=-1)
context = (lstm_out * attention_weights.unsqueeze(-1)).sum(dim=1)

print(f"Attention weights shape: {attention_weights.shape}")
print(f"Context vector shape:    {context.shape}")
print(f"\nAttention weights (which patches matter most):")
print(attention_weights[0].detach().numpy().round(3))
```

```
Attention weights shape: torch.Size([1, 49])
Context vector shape:    torch.Size([1, 512])

Attention weights (which patches matter most):
[0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02
0.02
 0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02
0.02
```

```
 0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.021 0.021
0.02
 0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02  0.02
0.02
 0.02 ]
```

All probablities same because havent trained model yet so treat every patch with equal probability

```
# Three classification heads
num_styles  = 27
num_artists = 125
num_genres  = 11

style_head  = nn.Linear(512, num_styles)
artist_head = nn.Linear(512, num_artists)
genre_head  = nn.Linear(512, num_genres)

#predictions
style_logits  = style_head(context)
artist_logits = artist_head(context)
genre_logits  = genre_head(context)

print(f"Context (input to heads): {context.shape}")
print(f"Style output:  {style_logits.shape}  → 27 style scores")
print(f"Artist output: {artist_logits.shape} → 125 artist scores")
print(f"Genre output:  {genre_logits.shape}  → 11 genre scores")
```

```
Context (input to heads): torch.Size([1, 512])
Style output:  torch.Size([1, 27])  → 27 style scores
Artist output: torch.Size([1, 125]) → 125 artist scores
Genre output:  torch.Size([1, 11])  → 11 genre scores
```

so overall Architecture:

Painting[1,3,224,224] (1 img, 3 color channel and 224x224 pixels) | (resnet50 cnn) [1, 2048, 7, 7] = 49 patches | (reshape) [1, 49, 2048] (sequence of oatches) | linear projection [1, 49, 512] (compressed sequence to save memory) | bideirecional lstm [1,49, 512] | attention 1,512 = single painting rep | 3 headers style, artist, genre

```
import torch
import torch.nn as nn
import torchvision.models as models

class ArtCRNN(nn.Module):
    def __init__(self, num_styles=27, num_artists=125, num_genres=11):
        super().__init__()

        # CNN
```

```python
        backbone = models.resnet50(weights="IMAGENET1K_V2")
        self.features = nn.Sequential(*list(backbone.children())[:-2])

        # Freeze early layers
        for param in list(self.features.parameters())[:-20]:
            param.requires_grad = False

        # Projection
        self.projection = nn.Linear(2048, 512)

        # Bidirectional LSTM
        self.lstm = nn.LSTM(
            input_size=512,
            hidden_size=256,
            num_layers=2,
            batch_first=True,
            bidirectional=True,
            dropout=0.5
        )

        # Classification heads
        self.style_head  = nn.Linear(512, num_styles)
        self.artist_head = nn.Linear(512, num_artists)
        self.genre_head  = nn.Linear(512, num_genres)

    def forward(self, x):
        # CNN
        features = self.features(x)

        # Reshape to sequence
        b, c, h, w = features.shape
        sequence = features.flatten(start_dim=2).permute(0, 2, 1)

        # Project
        sequence = self.projection(sequence)

        # LSTM
        lstm_out, _ = self.lstm(sequence)

        # Attention
        weights = torch.softmax(lstm_out.mean(dim=-1), dim=-1)
        context = (lstm_out * weights.unsqueeze(-1)).sum(dim=1)

        # Heads
        return {
            "style":  self.style_head(context),
            "artist": self.artist_head(context),
            "genre":  self.genre_head(context)
        }
```

```python
# Test it
model = ArtCRNN()
fake  = torch.randn(2, 3, 224, 224)
out   = model(fake)
print("Model outputs:")
for k, v in out.items():
    print(f"  {k}: {v.shape}")
print("\n Model built successfully")
```

```
Model outputs:
  style: torch.Size([2, 27])
  artist: torch.Size([2, 125])
  genre: torch.Size([2, 11])

 Model built successfully
```

```python
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
from PIL import Image

class WikiArtDataset(Dataset):
    def __init__(self, hf_dataset, transform=None):
        self.dataset   = hf_dataset
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        item  = self.dataset[idx]
        image = item["image"].convert("RGB")

        if self.transform:
            image = self.transform(image)

        return image, item["style"], item["artist"], item["genre"]

# Transforms
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# Split into train and validation
train_size = 400
```

```
val_size   = 100

train_data = WikiArtDataset(dataset.select(range(train_size)),
transform)
val_data   = WikiArtDataset(dataset.select(range(train_size,
train_size + val_size)), transform)

train_loader = DataLoader(train_data, batch_size=16, shuffle=True)
val_loader   = DataLoader(val_data,   batch_size=16, shuffle=False)

print(f"Training paintings:   {len(train_data)}")
print(f"Validation paintings: {len(val_data)}")
print(f"Training batches:     {len(train_loader)}")
print(f"Validation batches:   {len(val_loader)}")
```

```
Training paintings:   400
Validation paintings: 100
Training batches:     25
Validation batches:   7
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using: {device}")

model = ArtCRNN().to(device)

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss()

def train_epoch(model, loader, optimizer):
    model.train()
    total_loss = 0
    style_correct = 0
    total = 0

    for images, styles, artists, genres in loader:
        images  = images.to(device)
        styles  = styles.to(device)
        artists = artists.to(device)
        genres  = genres.to(device)

        optimizer.zero_grad()
        outputs = model(images)

        loss = (
            criterion(outputs["style"],  styles)  * 1.0 +
            criterion(outputs["artist"], artists) * 0.9 +
            criterion(outputs["genre"],  genres)  * 0.8
        )

        loss.backward()
        optimizer.step()
```

```
        total_loss += loss.item()
        style_correct += (outputs["style"].argmax(1) ==
styles).sum().item()
        total += len(images)

    return total_loss / len(loader), style_correct / total

def val_epoch(model, loader):
    model.eval()
    style_correct = 0
    total = 0

    with torch.no_grad():
        for images, styles, artists, genres in loader:
            images  = images.to(device)
            styles  = styles.to(device)
            outputs = model(images)
            style_correct += (outputs["style"].argmax(1) ==
styles).sum().item()
            total += len(images)

    return style_correct / total

# Train for 5 epochs
print("Starting training...\n")
for epoch in range(5):
    train_loss, train_acc = train_epoch(model, train_loader,
optimizer)
    val_acc = val_epoch(model, val_loader)
    print(f"Epoch {epoch+1}/5 | Loss: {train_loss:.3f} | Train Acc:
{train_acc:.1%} | Val Acc: {val_acc:.1%}")

print("\n Training complete")
```

```
Using: cuda
Starting training...

Epoch 1/5 | Loss: 9.223 | Train Acc: 20.0% | Val Acc: 34.0%
Epoch 2/5 | Loss: 7.460 | Train Acc: 22.8% | Val Acc: 12.0%
Epoch 3/5 | Loss: 6.569 | Train Acc: 35.8% | Val Acc: 33.0%
Epoch 4/5 | Loss: 5.957 | Train Acc: 38.8% | Val Acc: 25.0%
Epoch 5/5 | Loss: 5.360 | Train Acc: 46.8% | Val Acc: 39.0%

 Training complete
```

Loss decreasing as expected, training acc now at 46.8 beating the 25% baseline of always predicting impressionism, validation acc is jumpy cuz 100 is too small a validation set, should get smoother with more data. However, gap between train (46.8) and val (39) suggests slight overfitting

```python
from sklearn.metrics import classification_report
import numpy as np

#style names that actually appear in validation set
unique_classes = sorted(set(all_style_true))
present_names  = [style_names[i] for i in unique_classes]

print("STYLE CLASSIFICATION REPORT\n")
print(classification_report(
    all_style_true,
    all_style_preds,
    labels=unique_classes,
    target_names=present_names,
    zero_division=0
))
```

```
STYLE CLASSIFICATION REPORT

                       precision    recall  f1-score   support

          Art_Nouveau       0.00      0.00      0.00         5
              Baroque       0.00      0.00      0.00         6
               Cubism       0.00      0.00      0.00         1
        Expressionism       0.00      0.00      0.00         8
        Impressionism       0.77      0.50      0.61        34
Naive_Art_Primitivism       0.00      0.00      0.00         2
 Northern_Renaissance       0.00      0.00      0.00         8
   Post_Impressionism       1.00      0.17      0.29        12
              Realism       0.16      0.88      0.27         8
          Romanticism       0.30      0.86      0.44         7
            Symbolism       0.54      0.88      0.67         8
     Synthetic_Cubism       0.00      0.00      0.00         1

             accuracy                           0.39       100
            macro avg       0.23      0.27      0.19       100
         weighted avg       0.46      0.39      0.35       100
```

model leraned to predict common styles aka impressionism, realism, romanticism but ignores rare ones. Such as impressionism and symbolism having f1 high scores of 0.67 and 0.61. Some noteable takeaways: Post impressionism precision is 1 aka when it predicts it its alaways right however recall of 0.17 means misses actual 83% of post imp paintings, realism has recall 0.88 aka finds 88% reaism paintng but precison 0.16 means it calls out realism incorrevly

```python
import matplotlib.pyplot as plt
import torch.nn.functional as F

model.eval()
outliers = []
```

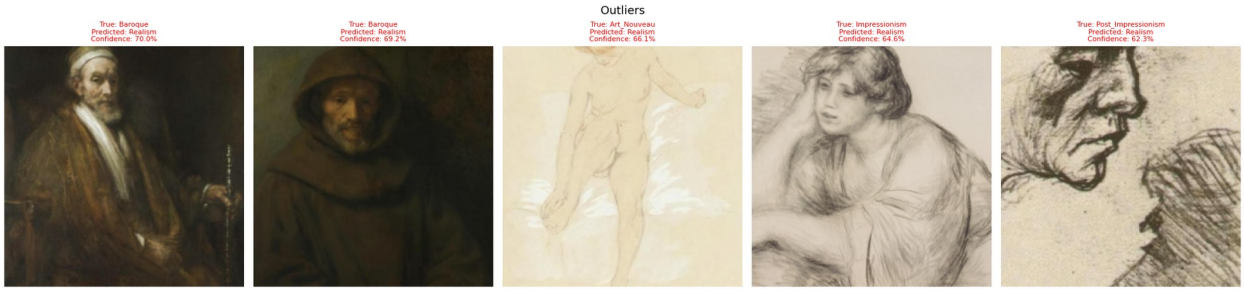
```python
with torch.no_grad():
    for images, styles, artists, genres in val_loader:
        images  = images.to(device)
        outputs = model(images)
        probs   = F.softmax(outputs["style"], dim=1)

        for i in range(len(images)):
            true_label = styles[i].item()
            pred_label = probs[i].argmax().item()
            confidence = probs[i].max().item()
            true_prob  = probs[i][true_label].item()

            # Outlier (confident but wrong)
            if pred_label != true_label and confidence > 0.4:
                outliers.append({
                    "image":      images[i].cpu(),
                    "true":       style_names[true_label],
                    "predicted":  style_names[pred_label],
                    "confidence": confidence,
                    "true_prob":  true_prob,
                })

outliers = sorted(outliers, key=lambda x: -x["confidence"])
print(f"Found {len(outliers)} outliers where model is confident but
wrong")
print("\nTop outliers:")
for o in outliers[:5]:
    print(f"  True: {o['true']:<25} Predicted: {o['predicted']:<25}
Confidence: {o['confidence']:.1%}")
```

```
Found 19 outliers where model is confident but wrong

Top outliers:
  True: Baroque                   Predicted: Realism
Confidence: 70.0%
  True: Baroque                   Predicted: Realism
Confidence: 69.2%
  True: Art_Nouveau               Predicted: Realism
Confidence: 66.1%
  True: Impressionism             Predicted: Realism
Confidence: 64.6%
  True: Post_Impressionism        Predicted: Realism
Confidence: 62.3%
```

predicts everthing as realism just like we saw in classification report or eval metrics as realism
0.88 recall means model calls almost everythin realism

```python
import torchvision.transforms as T

# Denormalize image to display it
```

```
def denormalize(tensor):
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3,1,1)
    std  = torch.tensor([0.229, 0.224, 0.225]).view(3,1,1)
    return (tensor * std + mean).clamp(0,1)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))

for i, outlier in enumerate(outliers[:5]):
    img = denormalize(outlier["image"])
    img = img.permute(1,2,0).numpy()

    axes[i].imshow(img)
    axes[i].set_title(
        f"True: {outlier['true']}\n"
        f"Predicted: {outlier['predicted']}\n"
        f"Confidence: {outlier['confidence']:.1%}",
        fontsize=8, color="red"
    )
    axes[i].axis("off")

plt.suptitle("Outliers", fontsize=13)
plt.tight_layout()
plt.show()
```



Observation:

painting 1 and 2: Dark background, heavy shadow, model has little baroque trainiing hence the confusion

Painting 3 looks like a plain figure drawing without any decorative elements as expected from art_nouveau

last 2 paintings do not give typical impressionism at all, as they are just pencil sketches without the significant brush work or light effects (I personally think theyve been mislabelled in the dataset and model might actually be right)

```
from sklearn.metrics import f1_score, cohen_kappa_score
import numpy as np

# Overall metrics
accuracy = (np.array(all_style_preds) ==
```

```
np.array(all_style_true)).mean()
macro_f1 = f1_score(all_style_true, all_style_preds, average="macro",
zero_division=0)
weighted_f1 = f1_score(all_style_true, all_style_preds,
average="weighted", zero_division=0)
kappa = cohen_kappa_score(all_style_true, all_style_preds)


print("   FINAL EVALUATION SUMMARY")

print(f"  Accuracy:         {accuracy:.1%}")
print(f"  Macro F1:         {macro_f1:.3f}")
print(f"  Weighted F1:      {weighted_f1:.3f}")
print(f"  Cohen's Kappa:    {kappa:.3f}")
print(f"  Outliers found:   {len(outliers)}")

print(f"""

1. Accuracy alone is misleading (25.8% baseline = always predict
Impressionism)
2. Macro F1 weights all 27 classes equally — penalises ignoring rare
styles
3. Weighted F1 reflects real distribution — dominated by common styles
4. Cohen's Kappa removes credit for random chance
5. Outlier analysis reveals dataset quality issues
""")
```

```
   FINAL EVALUATION SUMMARY
  Accuracy:         39.0%
  Macro F1:         0.190
  Weighted F1:      0.347
  Cohen's Kappa:    0.294
  Outliers found:   19


1. Accuracy alone is misleading (25.8% baseline = always predict
Impressionism)
2. Macro F1 weights all 27 classes equally — penalises ignoring rare
styles
3. Weighted F1 reflects real distribution — dominated by common styles
4. Cohen's Kappa removes credit for random chance
5. Outlier analysis reveals dataset quality issues
```

Final Evaluation & What I Would Improve

My model achieved 39% accuracy and a Macro F1 of 0.19 on 100 validation paintings. Cohen's Kappa of 0.29 shows it is better than random chance but has room for improvement.

The low Macro F1 (0.19) compared to Weighted F1 (0.35) confirms what the class imbalance analysis showed that the model performs well on common styles like Impressionism but completely ignores rare styles like Synthetic_Cubism and Baroque.

The outlier analysis found 19 confidently wrong predictions. Two of these (Impressionism paintings predicted as Realism) appear to be mislabels as the paintings are pencil sketches with no visible Impressionist characteristics.

What I would improve with more time:

- Train on the full 80,000 painting dataset instead of 500
- Use weighted loss to penalise errors on rare classes more heavily
- Train for more epochs with learning rate scheduling
- Add data augmentation to reduce overfitting